

[OVERVIEW](#) [PACKAGE](#) [CLASS](#) [USE TREE](#) [DEPRECATED](#) [INDEX](#) [HELP](#)[PREV CLASS](#) [NEXT CLASS](#) [FRAMES](#) [NO FRAMES](#) [ALL CLASSES](#)[SUMMARY: NESTED](#) | [FIELD](#) | [CONSTR](#) | [METHOD](#) [DETAIL: FIELD](#) | [CONSTR](#) | [METHOD](#)`compact1, compact2, compact3``java.util`

Interface Queue<E>

Type Parameters:

E - the type of elements held in this collection

All Superinterfaces:

`Collection<E>, Iterable<E>`

All Known Subinterfaces:

`BlockingDeque<E>, BlockingQueue<E>, Deque<E>, TransferQueue<E>`

All Known Implementing Classes:

`AbstractQueue, ArrayBlockingQueue, ArrayDeque, ConcurrentLinkedDeque, ConcurrentLinkedQueue, DelayQueue, LinkedBlockingDeque, LinkedBlockingQueue, LinkedList, LinkedTransferQueue, PriorityBlockingQueue, PriorityQueue, SynchronousQueue`

```
public interface Queue<E>
extends Collection<E>
```

A collection designed for holding elements prior to processing. Besides basic `Collection` operations, queues provide additional insertion, extraction, and inspection operations. Each of these methods exists in two forms: one throws an exception if the operation fails, the other returns a special value (either `null` or `false`, depending on the operation). The latter form of the insert operation is designed specifically for use with capacity-restricted `Queue` implementations; in most implementations, insert operations cannot fail.

Summary of Queue methods

	<i>Throws exception</i>	<i>Returns special value</i>
Insert	<code>add(e)</code>	<code>offer(e)</code>
Remove	<code>remove()</code>	<code>poll()</code>
Examine	<code>element()</code>	<code>peek()</code>

Queues typically, but do not necessarily, order elements in a FIFO (first-in-first-out) manner. Among the exceptions are priority queues, which order elements according to a supplied comparator, or the elements' natural ordering, and LIFO queues (or stacks) which order the elements LIFO (last-in-first-out). Whatever the ordering used, the *head* of the queue is that element which would be removed by a call to `remove()` or `poll()`. In a FIFO queue, all new elements are inserted at the *tail* of the queue. Other kinds of queues may use different placement rules. Every `Queue` implementation must specify its ordering properties.

The `offer` method inserts an element if possible, otherwise returning `false`. This differs from the `Collection.add` method, which can fail to add an element only by throwing an unchecked exception. The offer method is designed for use when failure is a normal, rather than exceptional occurrence, for example, in fixed-capacity (or "bounded") queues.

The `remove()` and `poll()` methods remove and return the head of the queue. Exactly which element is removed from the queue is a function of the queue's ordering policy, which differs from implementation to implementation. The `remove()` and `poll()` methods differ only in their behavior when the queue is empty: the `remove()` method throws an exception, while the `poll()` method returns `null`.

The `element()` and `peek()` methods return, but do not remove, the head of the queue.

The Queue interface does not define the *blocking queue methods*, which are common in concurrent programming. These methods, which wait for elements to appear or for space to become available, are defined in the `BlockingQueue` interface, which extends this interface.

Queue implementations generally do not allow insertion of `null` elements, although some implementations, such as `LinkedList`, do not prohibit insertion of `null`. Even in the implementations that permit it, `null` should not be inserted into a Queue, as `null` is also used as a special return value by the `poll` method to indicate that the queue contains no elements.

Queue implementations generally do not define element-based versions of methods `equals` and `hashCode` but instead inherit the identity based versions from class `Object`, because element-based equality is not always well-defined for queues with the same elements but different ordering properties.

This interface is a member of the [Java Collections Framework](#).

Since:

1.5

See Also:

[Collection](#), [LinkedList](#), [PriorityQueue](#), [LinkedBlockingQueue](#), [BlockingQueue](#), [ArrayBlockingQueue](#), [LinkedBlockingQueue](#), [PriorityBlockingQueue](#)

Method Summary

All Methods	Instance Methods	Abstract Methods
Modifier and Type	Method	Description
boolean	<code>add(E e)</code>	Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning <code>true</code> upon success and throwing an <code>IllegalStateException</code> if no space is currently available.
E	<code>element()</code>	Retrieves, but does not remove, the head of this queue.

<code>boolean</code>	<code>offer(E e)</code>	Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions.
<code>E</code>	<code>peek()</code>	Retrieves, but does not remove, the head of this queue, or returns <code>null</code> if this queue is empty.
<code>E</code>	<code>poll()</code>	Retrieves and removes the head of this queue, or returns <code>null</code> if this queue is empty.
<code>E</code>	<code>remove()</code>	Retrieves and removes the head of this queue.

Methods inherited from interface `java.util.Collection`

`addAll`, `clear`, `contains`, `containsAll`, `equals`, `hashCode`, `isEmpty`, `iterator`, `parallelStream`, `remove`, `removeAll`, `removeIf`, `retainAll`, `size`, `splititerator`, `stream`, `toArray`, `toArray`

Methods inherited from interface `java.lang.Iterable`

`forEach`

Method Detail

add

`boolean add(E e)`

Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and throwing an `IllegalStateException` if no space is currently available.

Specified by:

`add` in interface `Collection<E>`

Parameters:

`e` - the element to add

Returns:

`true` (as specified by `Collection.add(E)`)

Throws:

`IllegalStateException` - if the element cannot be added at this time due to capacity restrictions

`ClassCastException` - if the class of the specified element prevents it from being added to this queue

`NullPointerException` - if the specified element is null and this queue does not permit null elements

`IllegalArgumentException` - if some property of this element prevents it from being added to this queue

offer

`boolean offer(E e)`

Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions. When using a capacity-restricted queue, this method is generally preferable to `add(E)`, which can fail to insert an element only by throwing an exception.

Parameters:

`e` - the element to add

Returns:

true if the element was added to this queue, else false

Throws:

`ClassCastException` - if the class of the specified element prevents it from being added to this queue

`NullPointerException` - if the specified element is null and this queue does not permit null elements

`IllegalArgumentException` - if some property of this element prevents it from being added to this queue

remove

`E remove()`

Retrieves and removes the head of this queue. This method differs from `poll` only in that it throws an exception if this queue is empty.

Returns:

the head of this queue

Throws:

`NoSuchElementException` - if this queue is empty

poll

`E poll()`

Retrieves and removes the head of this queue, or returns null if this queue is empty.

Returns:

the head of this queue, or null if this queue is empty

element

E element()

Retrieves, but does not remove, the head of this queue. This method differs from [peek](#) only in that it throws an exception if this queue is empty.

Returns:

the head of this queue

Throws:

[NoSuchElementException](#) - if this queue is empty

peek

E peek()

Retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.

Returns:

the head of this queue, or null if this queue is empty

[Submit a bug or feature](#)

For further API reference and developer documentation, see [Java SE Documentation](#). That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to [license terms](#). Also see the [documentation redistribution policy](#). [Modify Cookie Preferences](#). [Modify Ad Choices](#).